



NYBÖRJARKURS • 5 TRÄFFAR

Elektronik & Programmering med Arduino

Fem träffar. Fem moduler. Ett fungerande larm.

FR0 Ånge • 13 april 2026 • FR0-lokalen

ELEG00 UNO
Basic Starter Kit

Fem träffar. Fem moduler.



MODUL 01

LED & krets

Digital output,
Ohms lag



MODUL 02

PWM & RGB

analogWrite,
färgblandning



MODUL 03

Digital input

Knapp, buzzer,
logik



MODUL 04

Analog input

Sensorer, Serial
Monitor



MODUL 05

Integration

Hackathon: bygg
larmet

Varje träff bygger vidare på den föregående.

INFÖR TRÄFF 1 · KLASSRUMSVKTYGET

Klassrummets viktigaste verktyg



GRÖN

Allt flyter på

+



RÖD

Jag behöver hjälp

TRÄFF 1 AV 5



LED & krets

Modul 1: **Digital output, Ohms lag**

DAGENS UPPLÄGG

Idag — två timmar.



00:00 — 00:15

Slutmålet & Arduinon

Vi tittar på det färdiga larmet och ramar in vad ett Arduino-kort egentligen är.



00:15 — 00:35

Krets, spänning, ström

Kort genomgång av vad en krets är och varför LED:en behöver en resistor.



00:35 — 01:10

Bygg kretsen & ladda upp Blink

Praktiskt: LED + resistor på breadboarden, första uppladdningen från IDE:n.



01:10 — 02:00

Din egen rytm & delresultat

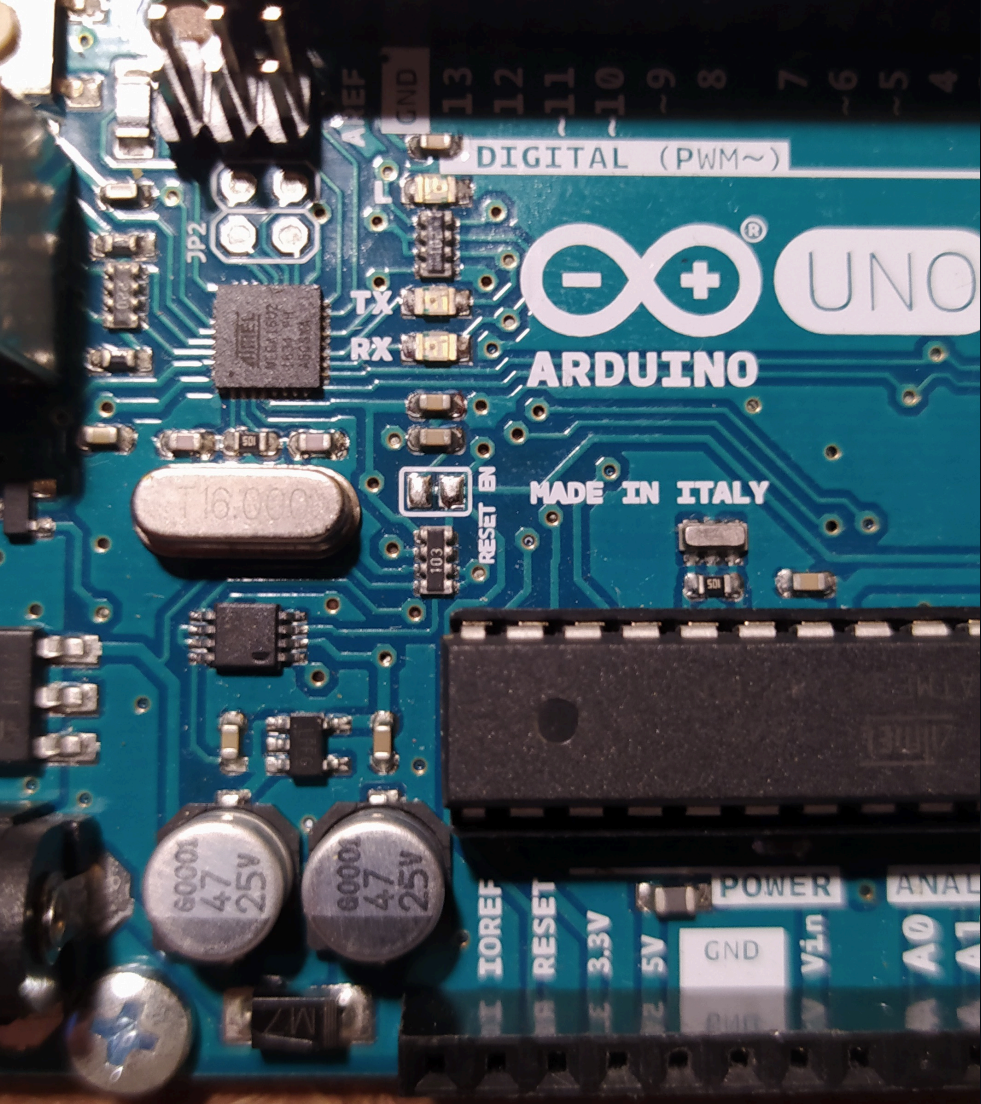
Fri övning: ändra delay-värdena till ett eget mönster. Alla visar upp.



SLUTMÅLET

Det här ska vi bygga.

Ett larm som känner av mörker, lyser upp som stämningsljus och tjuoter när någon rör det.



ARDUINO UNO R3

Mikrokontrollern.

En **mikrokontroller** — processor, minne och in-/utgångar på ett chip.

Kör ett enda program, helt förutsägbart.

Arduino UNO R3 • ATmega328P

16 MHz • 32 KB flash • 2 KB SRAM

Spänning, ström, GND.



SPÄNNING

Volt · V

Tryckskillnad mellan två punkter.
Arduinos **5V**-pinne ligger på 5 V
relativt GND.



STRÖM

Ampere · A

Laddning per sekund. En LED tål
ca **20 mA**. Mer → den brinner upp.



GND

0 V · referens

Nollpunkten all elektronik mäts
mot. Varje krets behöver en retur
till GND.

OHMS LAG



$$U = I \cdot R \quad \cdot \quad I = U / R \quad \cdot \quad R = U / I$$

LED-exempel: $(5 \text{ V} - 2 \text{ V}) / 15 \text{ mA} \approx 200 \Omega \rightarrow$ välj **220 Ω**

Varför U och I ? U från tyskans *Spannung* (spänning), I från franska *intensité du courant* (strömintensitet). Europeisk konvention sedan 1800-talet. Full förklaring av framspänningsfall i kompendiets Appendix F.

Kretsen — som ett vattensystem.

Strömmen går ut från en **utgångspinne** (t.ex. pin 13 när den är HIGH), genom komponenterna, och tillbaka till **GND**.

Utan bromsning: **LED:en brinner upp**.

Lösningen: en **strypventil** — en resistor på $220\ \Omega$ i serie.

MINNESREGEL

Ingen krets utan väg tillbaka till GND.
Strömmen måste kunna fullborda varvet —
annars händer ingenting.



pin 13 → LED (+)
LED (-) → $220\ \Omega$
 $220\ \Omega$ → GND

LED & resistor.

LED:en

- **Har polaritet.** Kör du den baklänges lyser den inte.
- **Långt ben = anod (+)** → till signalpinnen (pin 13).
- **Kort ben + platt kant = katod (-)** → till GND.
- **Tål ca 20 mA.** Mer → den brinner upp permanent.

Kör du den baklänges går den inte sönder — den lyser bara inte. Bara att vända.

Resistorn

- **Passiv komponent.** Ingen polaritet — koppla hur du vill.
- Värde läses på **färgringarna**.
- **220 Ω** = röd · röd · brun + guld (tolerans).



röd · röd · brun
 $2 \cdot 2 \cdot \times 10^1$
= **220 Ω**

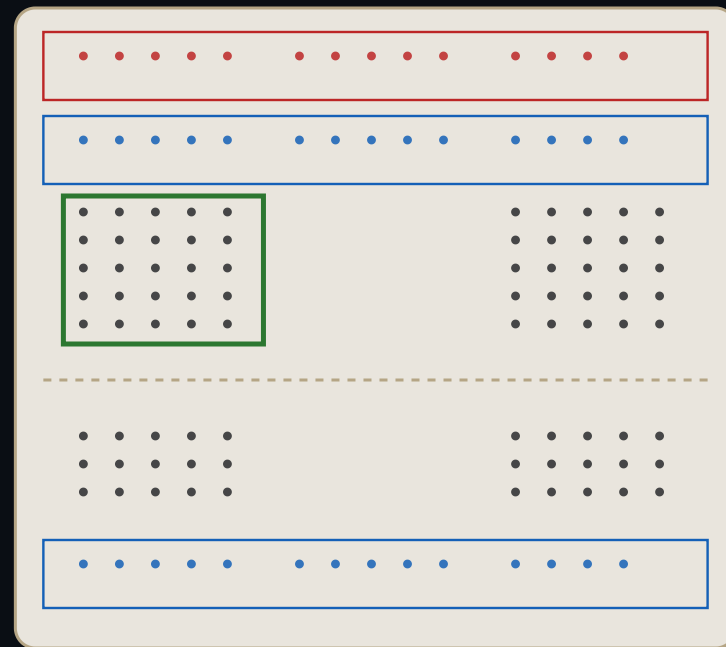
Fem hål = en nod.

En breadboard är bara en plastbit med metallklämmor inuti.

- Fem hål i en vågrät rad är internt hopkopplade — en **nod**.
- Raden bredvid är en **helt egen** nod.
- Gapet i mitten **bryter** förbindelsen.
- +/-skenorna längs sidorna går hela vägen.

NÄR LED:EN INTE LYSER

Kolla först att komponentbenet och kabeln faktiskt sitter på **samma** rad. 90 % av alla nybörjarfel är ett hål fel.



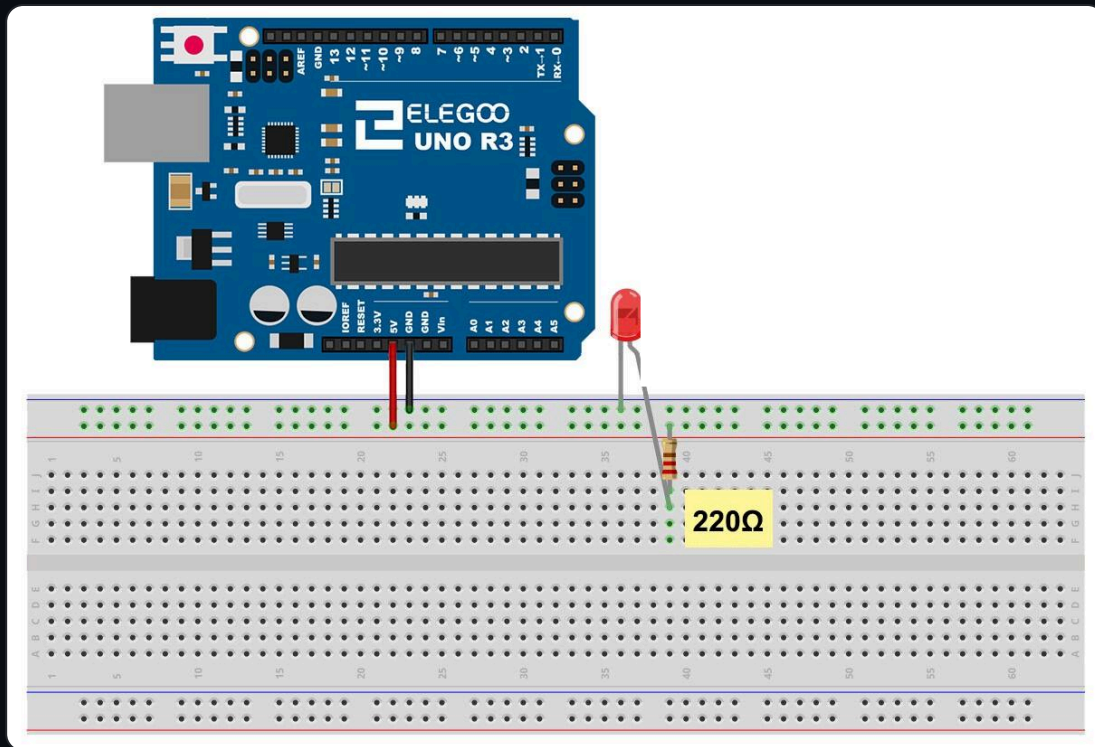
röd/blå rail · grön ruta = en nod (5 hål) · streckad linje = gapet bryter

Bygg kretsen.

- 01 Sätt LED:en på breadboarden. **Långt ben** i en ledig rad, **kort ben** i raden bredvid.
- 02 Kabel från **pin 13** på Arduinon → samma rad som långa benet.
- 03 **220 Ω -resistor** (röd-röd-brun). Ett ben i samma rad som korta LED-benet. Andra benet i valfri ledig rad.
- 04 Kabel från resistorns andra ben → **GND** på Arduinon.
- 05 Ladda upp **File** → **Examples** → **01.Basics** → **Blink**. Lampan ska börja blinka.

Elegoo-manualen, Lesson 3 (sid 37–43). Kopplingsschema på sid 42.

Så här ska det se ut.



D13 → LED långt ben · LED kort ben → 220 Ω → GND

setup() och loop() .

void setup()

Körs en gång när Arduinon startar eller när ni laddar upp. Används för konfiguration: vilka pinnar är utgångar, hastighet på Serial, etc.

void loop()

Körs om och om igen, så länge kortet har ström. Här bor logiken: läs, reagera, vänta, upprepa.

Regel: exakt en setup() och en loop() per Arduino-program ("sketch"). Aldrig fler.

```
void setup() {  
  // Körs en gång vid uppstart.  
  pinMode(13, OUTPUT);  
}  
  
void loop() {  
  // Körs om och om igen.  
  digitalWrite(13, HIGH);  
  delay(1000);  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```

TRE KOMMANDON

Hela Blink — på tre rader.

```
pinMode(13, OUTPUT);
```

"Pin 13 är en **utgång**." Säger åt Arduinon att den ska skicka ut ström på den pinnen. Körs i `setup()`.

```
digitalWrite(13, HIGH);
```

"Sätt pin 13 till **5V**." HIGH = tänd (5 V), LOW = släckt (0 V). Körs i `loop()`.

```
delay(1000);
```

"Vänta **1000 millisekunder**." 1000 ms = 1 sekund. Arduinon pausar allt annat under tiden.

Med bara dessa tre kan ni få en lysdiod att blinka i vilken rytm som helst.

UPPGIFT

Din egen rytm.

Ändra `Blink` till ett mönster du själv väljer.

- **SOS-rytm:** · · · — — — · · ·
- **Ditt eget tempo** — snabb, långsam, oregelbunden
- **Hjärtslag** — två snabba, sen paus

Tips: lägg tider i `const int`-variabler så slipper du ändra många siffror varje gång.

```
const int KORT = 200;
const int LANG = 600;

void setup() {
  pinMode(13, OUTPUT);
}

// förenklad timing, inte strikt CW
void loop() {
  // S
  blink(KORT); blink(KORT); blink(KORT);
  delay(400);
  // O
  blink(LANG); blink(LANG); blink(LANG);
  delay(400);
  // S
  blink(KORT); blink(KORT); blink(KORT);
  delay(2000);
}

void blink(int tid) {
  digitalWrite(13, HIGH); delay(tid);
  digitalWrite(13, LOW); delay(tid);
}
```

DELRESULTAT • TRÄFF 1

KLART

Ni har byggt en blinkare.

Er första rad kod som styr den fysiska världen.



Er första krets

LED + resistor + GND



Er första sketch

setup, loop, tre kommandon



Er egen rytm

kod som styr fysiska världen

Ett varv runt rummet — visa upp ditt mönster för grannen.

NÄSTA TRÄFF



Nästa gång...

Ni kan tända en lampa i en egen rytm.
Nästa steg: blanda **färg** ur rött, grönt och blått.

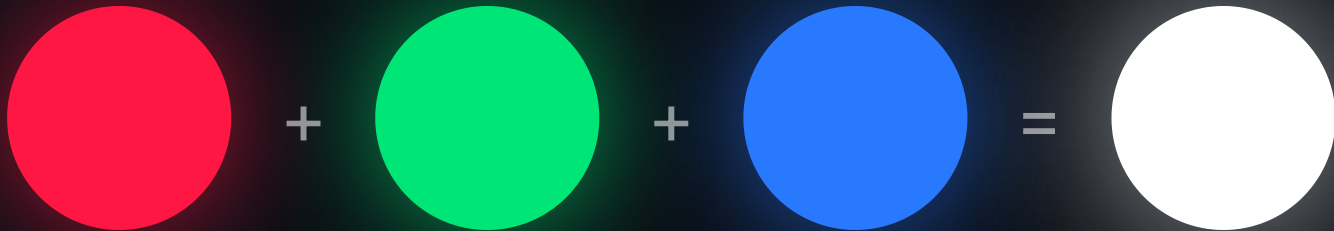
TRÄFF 2 AV 5



PWM & RGB

Modul 2: **analogWrite** & färgblandning

Alla färger, av bara tre.



Skärmen ni tittar på just nu har **miljoner pixlar**, och varje pixel är bara en röd, en grön och en blå punkt.

Idag programmerar vi **en egen pixel**.

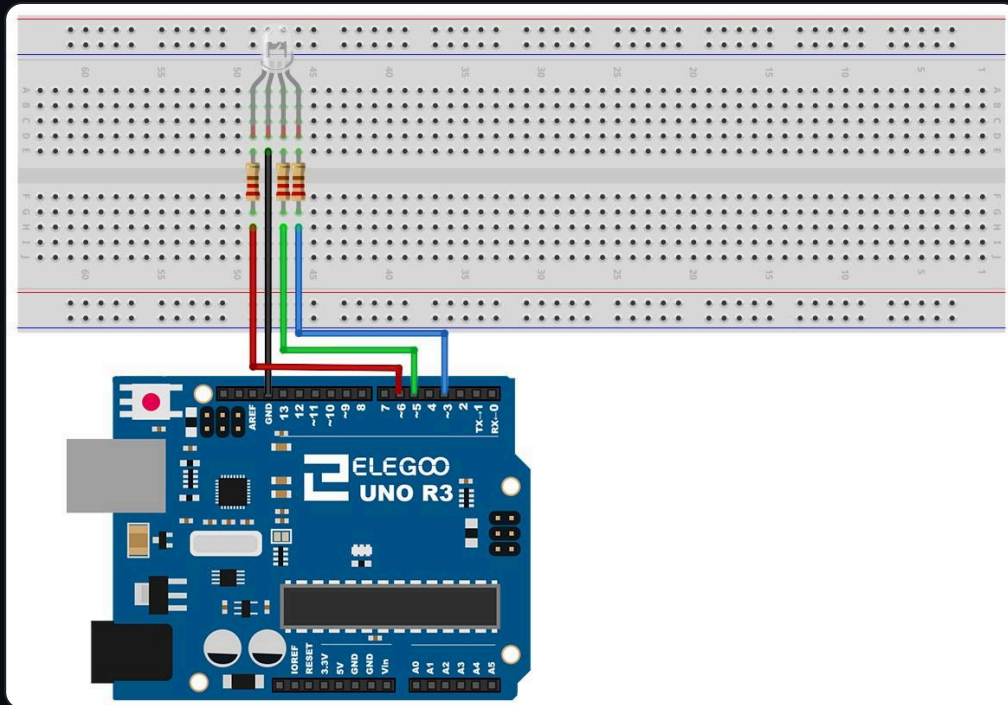
VIKTIGT — RGB-LED:EN

Common Cathode. 4 ben i rad.

Ordning från platta sidan: Röd · **Katod** · Grön · Blå

Katoden är **längst**, sitter **andra från platta sidan**, och går till **GND**.

Koppla RGB-LED:en.



R → D6 · G → D5 · B → D3 · katod → GND · varje färg via egen 220 Ω.

Gör detta först: dra en kabel från Arduinons GND till --skenan på breadboarden. Då har alla komponenter en minus-väg.

analogWrite •

`digitalWrite` kunde bara två saker: PÅ (HIGH) eller AV (LOW).

`analogWrite` tar ett tal från 0 till 255.

0 = helt av. 255 = helt på.

Allt däremellan är PWM — snabb blinkning som ögat blandar ihop.

MER OM PWM I KOMPENDIET

Kapitel 2 förklarar duty cycle och varför bara pinnar märkta ~ (3, 5, 6, 9, 10, 11) stöder `analogWrite`.

```
// Kärnan – full sketch i kompendiet
analogWrite(ledR, 200); // röd: hög
analogWrite(ledG, 0);   // grön: av
analogWrite(ledB, 200); // blå: hög
// → lila
```

*Pins deklarerade som `const int` i `setup`,
se kompendiet kapitel 2 för hela skisset.*

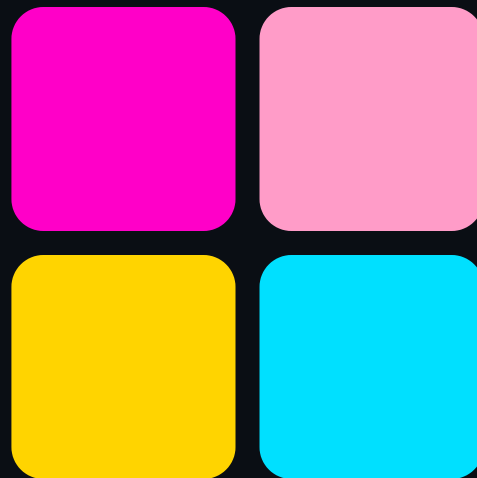
Hitta färgen.

Experimentera med värden 0–255 på varje kanal.
Ladda upp, titta, justera.

- **Lila** — röd + blå, ingen grön
- **Gammelrosa** — mycket röd, lite blå, knapp grön
- **Skolgul** — full röd, lagom grön, ingen blå
- **Cyan** — ingen röd, full grön + blå

REGELN

Grön lapp = du hittade en färg du gillar. Röd lapp = du fastnat på något.



EFTER TRÄFF 2

Ni har en pixel.



Ni kan blanda *alla färger* ur tre kanaler, och ni har lärt Arduinon att leverera något *mellan* helt av och helt på.

Nästa gång: vi lyssnar. Knapp + buzzer.

TRÄFF 3 AV 5



Digital input

Modul 3: **Knapp, buzzer, logik**

const int och int.

I morse har ni redan använt `const int` —
för pin-nummer.

Nu behöver ni också `int` (utan `const`) —
för värden som **ska kunna ändras under
tiden programmet kör**: räknare, tillstånd,
sensormätningar.

TUMREGEL

`const int` = sätts en gång, aldrig
mer.

`int` = kan ändras.

`bool` = sant eller falskt.

```
const int knappPin = 9;    // ändras aldrig  
int tryckCount = 0;        // kan öka  
bool larmPaslaget = false; // kan togglas
```

*Full förklaring av datatyper,
scope och operatorer i kompendiet,
Appendix A (syntax-grammatik).*

Input $\neq$ Output.

Hittills har Arduinon bara skickat ström ut (OUTPUT).
Nu läser den av ett tillstånd **in** (INPUT).

```
pinMode(knappPin, INPUT_PULLUP);    // intern pullup-resistor  
int nivå = digitalRead(knappPin);    // LOW = tryckt, HIGH = släppt
```

VARFÖR PULLUP?

En pinne som bara "lyssnar" utan koppling till plus eller minus *flyter* — läsningen blir slumpmässig. **INPUT_PULLUP** låter Arduinon själv dra pinnen till HIGH. När du trycker dras den till GND (LOW). Därför: **tryckt = LOW**, **släppt = HIGH**.

if / else .

Läs ett värde → jämför → agera olika beroende på resultatet.

Operatorerna: `=` lika, `≠` olika, `<` mindre, `>` större.

VANLIGASTE NYBÖRJARFELET

`=` tilldelar värde.

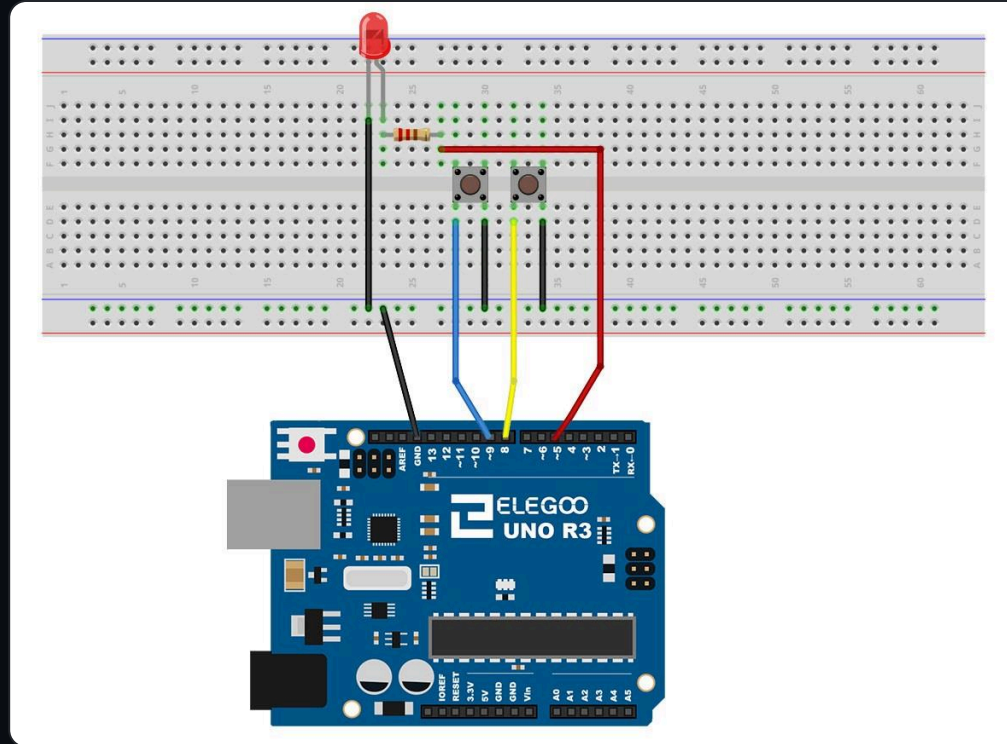
`=` jämför.

Glöm inte andra likhetstecknet.

```
if (digitalRead(knappPin) == LOW) {  
  // knappen är tryckt  
  digitalWrite(LED_BUILTIN, HIGH);  
} else {  
  // släppt  
  digitalWrite(LED_BUILTIN, LOW);  
}
```

Fler operatorer (`&&`, `||`, `!`)
och kedjade `if/else` i kompendiets Appendix A.

Koppla knappen.



Vi använder bara knapp A → D9 · andra benet till GND. Ingen pulldown-resistor — `INPUT_PULLUP` sköter det internt.

Reagera på flanken.

En knapp som **hålls nere** skulle toggla 50 gånger per sekund. Resultat: larmet blinkar mellan av/på som en stroboskop.

Lösningen: agera inte på att knappen *är* nere. Agera på att den **just nu gick från HIGH till LOW** — flanken.

MÖNSTRET

Spara förra värdet. Jämför med nuvarande. Om det ändrats åt rätt håll → det är en flank → agera en gång.

```
bool larmPaslaget = false;
int lastState = HIGH;

void loop() {
  int state = digitalRead(knappPin);
  if (state == LOW && lastState == HIGH) {
    // just nu tryckt – toggla
    larmPaslaget = !larmPaslaget;
  }
  lastState = state;
  delay(10); // mot studs
}
```

En varning först.

⚠ EXTREMT VIKTIGT

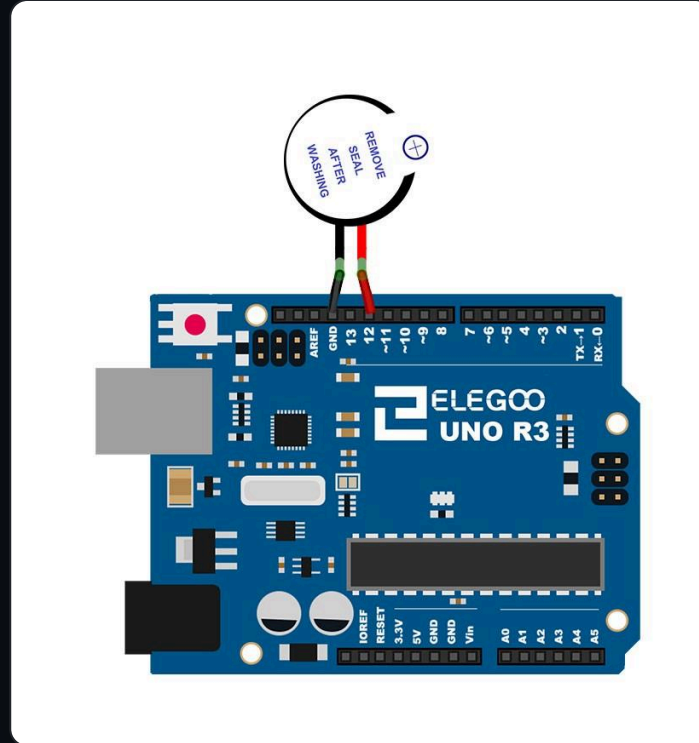
Dra absolut inte av den lilla klisterlappen
"Remove after washing"
på er buzzer.

Gör ni det kommer era öron att **blöda** och jag kastar ut er. Den är vår ljuddämpare.



Active buzzer • sticker stannar

Koppla buzzern.



Långt ben (+) → D12 · kort ben → GND. Rakt in i Arduino-headerna. Ingen breadboard, ingen resistor.

Läs knapp. Styr buzzer.

Första riktiga "input → output"-programmet.
Knappen ner → buzzern tjuiter. Släpp → tyst.

Komponenterna ni redan har:

- `pinMode(knappPin, INPUT_PULLUP)` från förra sliden
- `pinMode(buzzerPin, OUTPUT)` — nytt idag
- `if/else` från if/else-sliden
- `digitalRead` läser, `digitalWrite` styr

FULL SKETCH

Komplett körbar kod i kompendiets kapitel 3, inklusive kommentarer och

```
// Kärnan i loop()
if (digitalRead(knappPin) == LOW) {
    digitalWrite(buzzerPin, HIGH);
} else {
    digitalWrite(buzzerPin, LOW);
}
```

*Active buzzer: HIGH = pip,
LOW = tyst.
Ingen tone() behövs.*

EFTER TRÄFF 3

Arduinon lyssnar.

Ni har byggt Arduinons första **sinnesorgan** — en knapp — och gett den en **röst** — buzzern.

En läser, den andra reagerar.

*Nästa gång: **analoga sensorer**. Mörker, rörelse, mätvärden. Serial Monitor.*

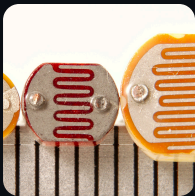
TRÄFF 4 AV 5



Analog input

Modul 4: **Sensorer & Serial Monitor**

Knappen var digital. Världen är analog.



Fotocell (LDR)

Mäter ljus: 0-1023

50 k Ω i mörker · 500 Ω i solljus

Och en till: **tilt-sensorn**. Den är lika digital som knappen — en kula som kortsluter vid lutning. Den kopplar vi efter fotocellen.

Knappen var **digital**: PÅ eller AV.

Ljuset är **analogt**: halvmörkt, ljust, mitt emellan.

```
int ljus = analogRead(A0); // 0 - 1023
```

GÖR DETTA NU

Bygg **spänningsdelaren**: fotocell + 1k Ω → pin A0

Spänningsdelaren.

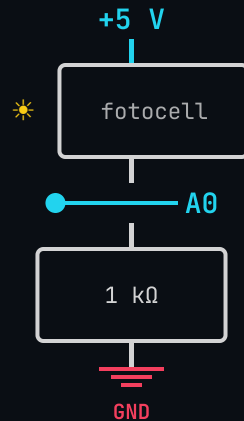
Minns vattenröret från Modul 1: **5 V** är **kranen**,
GND är **utloppet**, en resistor är en strypventil.

Koppla nu **två** ventiler i serie. Flödet (strömmen) är
lika stort genom båda — men trycket (våra 5 V)
delas upp. Hårdare ventil → större tryckfall över
just den.

Byt den övre ventilen mot en **fotocell**. Mörker →
högt motstånd → mellanpunkten **sjunker**. Ljus →
lågt motstånd → **stiger**.

A0 är en tryckmätare för mellanpunkten.

`analogRead(A0)` ger ett tal **0–1023** — Arduinos
10-bits linjal över 0–5 V.



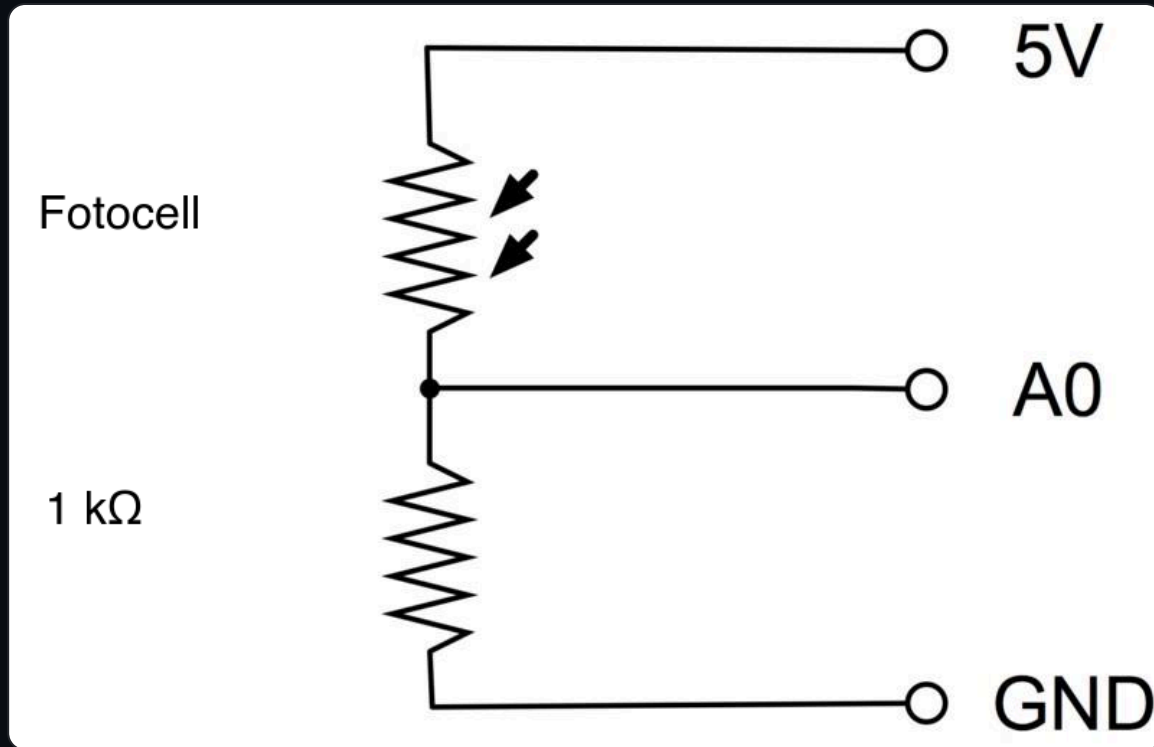
TYPISKA VÄRDEN · 1 kΩ

Hand över → **20–100**

Rumsljus → **150–400**

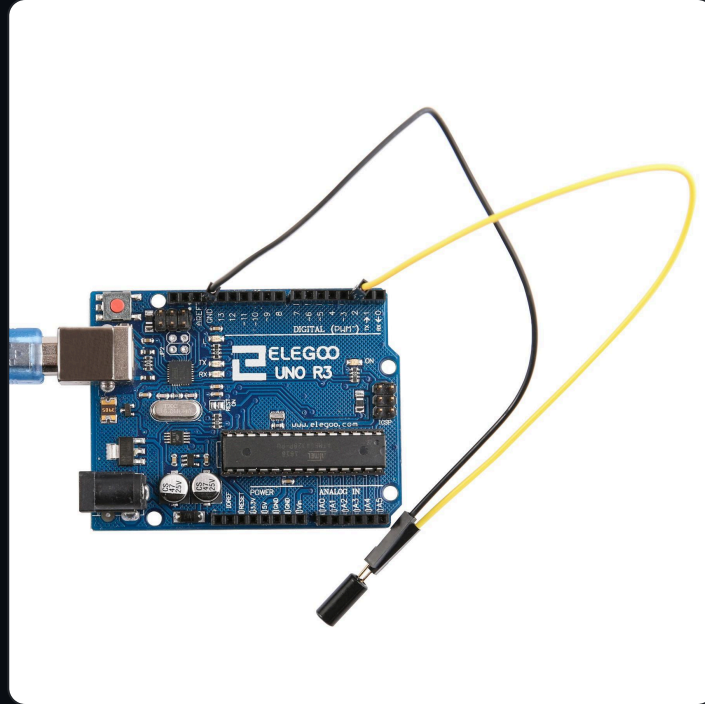
Lampa nära → **600–900**

Koppla fotocellen.



Spänningsdelare: 5V → fotocell → A0 → 1 kΩ → GND. Mörker ger lågt värde, ljus ger högt värde.

Koppla tilt-sensorn.



Rakt in i Arduino-headerna med F-M DuPont-kablar: **ena benet** → D2, **andra benet** → GND. Kräver ingen breadboard. Digital sensor — öppen/sluten, precis som en knapp. Läses med `digitalRead`, inte `analogRead`.

FELSÖKNING

Serial Monitor.

Hur ser vi vilka värden Arduinon läser?

Vi låter den skriva ut dem till datorn över USB.

Öppna **förstoringsglaset** uppe till höger i Arduino IDE.

```
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  int ljus = analogRead(A0);  
  Serial.println(ljus);  
  delay(100);  
}
```

Hitta din tröskel.

Printa **både** ljus och tilt till Serial Monitor.
Jämför siffrorna mot omgivningen.

- Täck fotocellen — vilket värde?
- Lys på den — vilket värde?
- Håll tilt-sensorn upprätt, sen lutad — HIGH
→ LOW?

HITTA ETT TAL ATT KALLA "MÖRKT"

Testa några olika tröskel-värden (300?
400? 500?) tills larmet reagerar lagom

```
void loop() {  
  int ljus = analogRead(A0);  
  int tilt = digitalRead(2);  
  Serial.print("ljus=");  
  Serial.print(ljus);  
  Serial.print(" tilt=");  
  Serial.println(tilt);  
  delay(200);  
}
```

Full sketch med setup() i kompendiet, kapitel 4.

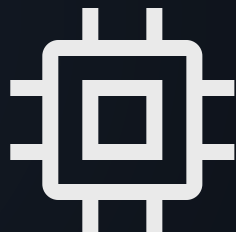
EFTER TRÄFF 4

Arduinon känner världen.

Ni kan läsa **ljus** med en fotocell, **lutning** med en tilt-sensor, och **titta in** i Arduinons hjärna via Serial Monitor.

*Nästa vecka: **hackathon**. Ni sätter ihop allt till ett fungerande tjuvlarm. Kom hungriga.*

TRÄFF 5 AV 5



Integration

Modul 5: **Hackathon** — bygg larmet

HACKATHON

Ingenjörsuppgiften.

Ingen manual att kopiera. Ni har alla delar. **Sätt ihop dem.**



INPUT

Knapp · LDR · Tilt



LOGIK

Arduino & kod



OUTPUT

RGB-LED · Buzzer

Regler: Knappen togglar larm-läge. Larm PÅ + tilt röd = tjut + rött blink. Larm AV + mörkt = stämningsljus.

Pin-tilldelning

Samma pinnar som ni använt i varje lektion.

```
const int knappPin = 9;    // Lesson 5
const int tiltPin   = 2;    // Lesson 7
const int ldrPin    = A0;   // Lesson 10
const int buzzerPin = 12;   // Lesson 6
const int ledR      = 6;    // Lesson 4
const int ledG      = 5;    // Lesson 4
const int ledB      = 3;    // Lesson 4

bool larmPaslaget = false;
```

```
void setup() {
  Serial.begin(9600);
  pinMode(knappPin, INPUT_PULLUP);
  pinMode(tiltPin, INPUT_PULLUP);
  pinMode(buzzerPin, OUTPUT);
  pinMode(ledR, OUTPUT);
  pinMode(ledG, OUTPUT);
  pinMode(ledB, OUTPUT);
}

void loop() {
  // 1. Läs inputs
  // 2. Flank → toggla larm
  //    (se Modul 3)
  // 3. Bestäm utfall:
  //    larm + tilt → tjut
  //    !larm + mörkt → ljus
  delay(10);
}
```

EFTER FEM TRÄFFAR

Tack.

Ni har byggt ett system som **läser av omvärlden**
och **reagerar på den**.

Det är grunden i inbyggda system.

FR0 Ånge • 2026